

A Python-Based Approach to Animate Geometry and Data

Shubham Tiwari

Supervisor: Dr. J. V. Ramani

Co-Supervisor: Ms. Ishver Rani Manchanda

Abstract

Mathematical concepts are traditionally communicated through symbolic expressions and static diagrams. While such representations are mathematically precise, they often fail to illustrate the dynamic nature of geometric transformations and functional behavior. This work proposes a Python-based computational approach for animating mathematical structures, with the aim of improving conceptual understanding through visual and dynamic representations of mathematical motion.

This chapter primarily focuses on two-dimensional geometric and analytical visualizations. Using Python libraries such as NumPy and Matplotlib, several classical mathematical problems are animated, including loci of points, equivalent norms, and the classical sliding ladder problem from coordinate geometry. In these examples, implicit equations are converted into parametric laws, allowing us to simulate spatial motion and transformations smoothly over discrete time steps. This approach demonstrates how mathematical animation can serve both as an educational tool and as a computational method for exploring geometry. The implementation of advanced three-dimensional geometries and motions is allocated to the future scope of this project.

Contents

1	Introduction: Escaping the Static Chalkboard	2
1.1	The Problem with Traditional Mathematics	2
1.2	What is a Shape? What is Animation?	2
1.3	Translating Math to Code: The Animation Engine	3
1.3.1	Breaking Down the Code	4
1.4	The Power of Parametrization	4
1.4.1	The Sliding Ladder and the Cat	5
1.5	The Concept of Equivalent Norms	7
1.6	The l_p Unit Ball: From Diamonds to Squares	7
1.7	The Parametrization Strategy	7
1.8	Implementation: The Norm Animation Engine	8
1.8.1	Understanding the Code Logic	9
1.9	Plotting Implicit Functions: The Ellipse	10
1.10	Stochastic Geometry: The 2D Random Walk	13
1.11	Dynamic Transformations: Rotating a Curve	15
1.12	Further Explorations and Open Source Contributions in 2d	17
1.13	Project Environment and Dependency Management using UV	18
1.13.1	Workflow 1: Cloning and Running the Existing Project	18
1.13.2	Workflow 2: Starting a New Animation Project from Scratch	18
1.14	Expanding into the Third Dimension	20
1.15	Official Code Repository	20

Chapter 1

Introduction: Escaping the Static Chalkboard

1.1 The Problem with Traditional Mathematics

Think back to your first geometry class. You likely learned about circles, parabolas, and lines by staring at static, motionless diagrams drawn on a flat chalkboard. The teacher wrote down equations, plotted a few points, and asked you to "visualize" what happens when variables change.

This static approach is why so many students struggle to build mathematical intuition. Mathematics, especially coordinate geometry and calculus, is inherently dynamic. It is about motion, transformation, and rates of change. When we freeze these concepts onto a piece of paper, we lose the magic. We cannot see how the math actually behaves.

But look around you. The animated movies you watch, the video games you play, and the physics simulations used by engineers—all of these are built entirely on mathematics. Underneath every smooth animation is a set of geometric equations evolving over time. The beautiful truth is that you do not need a supercomputer or a PhD to create these animations. All you need is standard, first-or-second-year college mathematics, a basic understanding of computer logic, and a framework to translate your thoughts into code.

1.2 What is a Shape? What is Animation?

Before we start coding, we must understand how a computer "sees" geometry. In pure mathematics, a circle is a continuous, infinite set of points. But computers cannot process infinity.

To a computer, a shape is simply an approximation. It is a finite list of discrete points connected by straight, invisible lines. If we generate enough points and place them close together, our eyes are tricked into seeing a perfectly smooth curve.

So, what is animation? Animation is simply moving these points over time. Instead of giving a point a fixed coordinate like $(2,3)$, we give it coordinates that are functions of time, like $(x(t),y(t))$. By calculating the position of the points at $t = 1$, then clearing the screen and calculating them at $t = 2$, $t = 3$, and so on rapidly, we create the illusion of fluid motion.

1.3 Translating Math to Code: The Animation Engine

To make this happen, we use the Python programming language [1]. We rely on two main libraries: NumPy (which instantly calculates the math for thousands of points) and Matplotlib [2] (which draws the points on the screen like a flipbook).

Let us look at the standard template used to animate mathematical functions. The official structure of this code relies heavily on Matplotlib's `FuncAnimation` module [2].

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from matplotlib.animation import FuncAnimation
4
5 # 1. Setup the figure and axis
6 fig, ax = plt.subplots()
7 ax.set_xlim(-5, 5)
8 ax.set_ylim(-5, 5)
9 ax.set_aspect('equal') # Keeps geometry undistorted
10
11 # 2. Create an empty line object to update later
12 line, = ax.plot([], [], 'b-', lw=2)
13
14 # 3. Initialization function (draws the blank canvas)
15 def init():
16     line.set_data([], [])
17     return line,
18
19 # 4. The Update Function (The Engine of Motion)
20 def update(frame):
21     # Mathematics happen here!
22     # 'frame' acts as our time variable 't'
23     x = np.linspace(-5, 5, 100) # Our discrete points
24     y = np.sin(x + frame/10)     # A moving sine wave
25
26     line.set_data(x, y)         # Give the new data to the plot
27     return line,
```

```

28
29 # 5. Run the animation
30 ani = FuncAnimation(fig, update, frames=200, init_func=init, blit=
    True, interval=30)
31 plt.show()

```

Listing 1.1: The General Matplotlib Animation Pipeline [2]

1.3.1 Breaking Down the Code

Let us explain exactly how this engine works, parameter by parameter:

- **Importing Libraries** (Lines 1-3): We bring in NumPy to handle arrays of numbers efficiently, and Matplotlib's `FuncAnimation` which acts as our movie projector.
- **`line, = ax.plot()`**: This creates a blank "actor" on our stage. It has a color (blue) and a line width (`lw=2`), but no data yet.
- **`def update(frame)`**: This is the heartbeat of the animation. The projector calls this function repeatedly. The `frame` variable is just an integer that goes up (0,1,2...). We use it as our "Time" variable to shift the math forward.
- **`line.set_data(x, y)`**: Once we calculate our new math for the current frame, we inject it into our line object.
- **`FuncAnimation` Parameters** [2]:
 - **`frames=200`**: Tells the animation to run for 200 steps before stopping or looping.
 - **`interval=30`**: The delay between frames in milliseconds. If you want the animation to run faster, decrease this to 10. For slow motion, increase it to 100.
 - **`blit=True`**: A rendering optimization. It tells the computer to only redraw the parts of the screen that have actually changed, making the animation incredibly smooth.

1.4 The Power of Parametrization

Now that we have the engine, we have to feed it the right fuel. In high school, we usually write equations in two ways:

1. Implicitly: $x^2 + y^2 = R^2$ (The relationship is mixed together).

2. Explicitly: $y = \sqrt{R^2 - x^2}$ (Solving for y).

Both of these are terrible for computer animation. Implicit equations do not give us a step-by-step way to generate points. Explicit equations often fail when curves bend backwards (like circles, which require $\pm$ square roots, causing broken lines in code).

The ultimate solution is Parametrization. Instead of forcing X and Y to interact directly, we introduce a third, independent variable (a parameter, often t or θ). We give X its own rule based on t , and Y its own rule based on t .

Parametrization does minimal work for maximum output. By just changing a single parameter t from 0 to 2π , X and Y gracefully dance across the screen to form complex shapes without ever breaking.

1.4.1 The Sliding Ladder and the Cat

To see why parametrization is vital, let us look at a classic physics problem often found in foundational coordinate geometry texts like Loney's *The Elements of Coordinate Geometry* [3]. Imagine a ladder of length L leaning against a vertical wall. Suddenly, the base slips, and the ladder slides down the wall until it hits the floor.

Now, imagine a cat is sitting exactly in the middle of this falling ladder. What path does the cat trace as it falls?

If you try to solve this with implicit algebra, it becomes an absolute nightmare of shifting coordinate systems. But with parametrization [3], it becomes beautiful and intuitive. Let the angle the ladder makes with the floor be θ . As the ladder falls, θ goes from 90° ($\pi/2$) down to 0° .

The top of the ladder on the y -axis is at $y = L \sin(\theta)$.

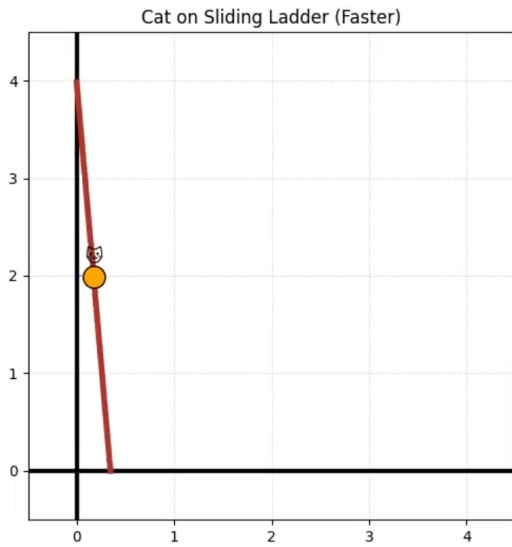
The bottom of the ladder on the x -axis is at $x = L \cos(\theta)$.

Since the cat is exactly in the middle, its coordinates are simply the averages:

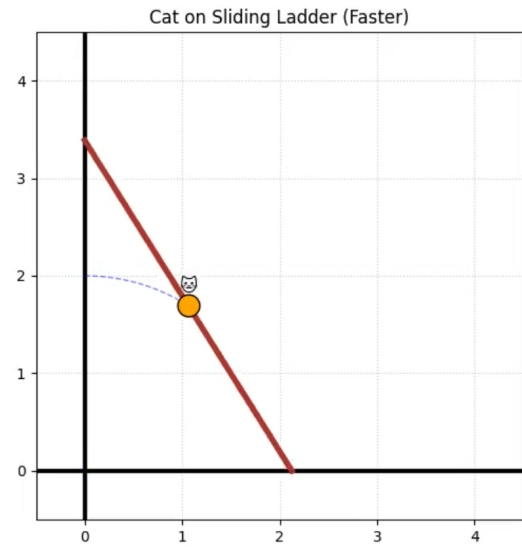
$$X_{cat}(\theta) = \frac{L}{2} \cos(\theta) \tag{1.1}$$

$$Y_{cat}(\theta) = \frac{L}{2} \sin(\theta) \tag{1.2}$$

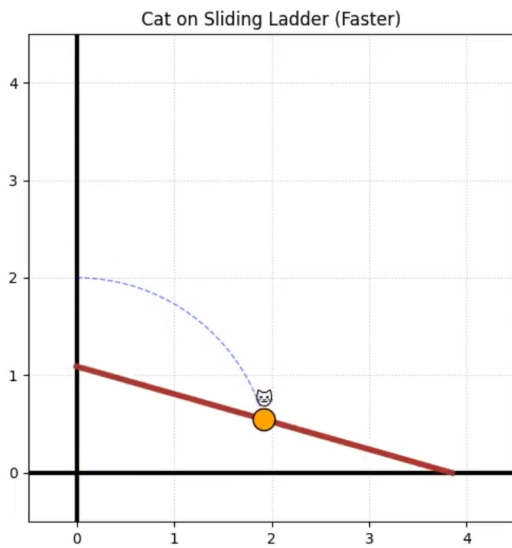
Look closely at those equations. They are the exact parametric equations of a circle with a radius of $L/2$! Even though the ladder is sliding in a straight line down and a straight line across, the cat traces a perfect circular arc through the air. This is the profound visual intuition that parametrization unlocks.



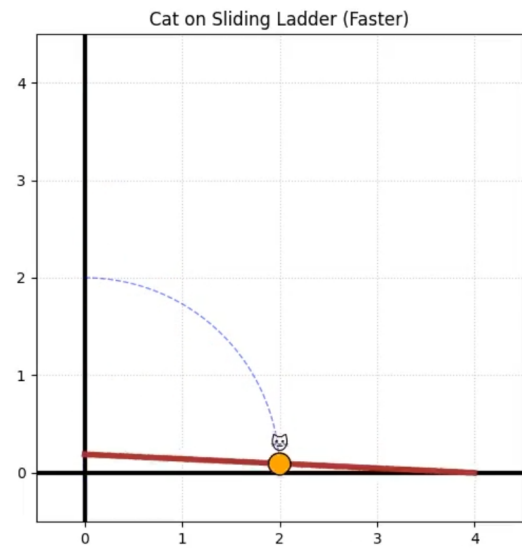
(a) Frame 1: Ladder starts sliding.



(b) Frame 2: Cat's path begins.



(c) Frame 3: Circular arc becomes visible.



(d) Frame 4: Ladder hits the floor.

Figure 1.1: The Cat on the Sliding Ladder: A visual progression over time.

Interactive Code Available!

You can view, run, and experiment with the full Python source code for the sliding ladder animation and other geometries in the official GitHub repository:

[Click here to view: cat_at_middle_ladder.py](https://github.com/Shubham18024/Python-Mathematical-Animation/blob/main/2d-anim/scripts/cat_at_middle_ladder.py)

[https://github.com/Shubham18024/Python-Mathematical-Animation/
blob/main/2d-anim/scripts/cat_at_middle_ladder.py](https://github.com/Shubham18024/Python-Mathematical-Animation/blob/main/2d-anim/scripts/cat_at_middle_ladder.py)

1.5 The Concept of Equivalent Norms

In our journey from basic geometry to advanced mathematics, we encounter the concept of a norm. A norm is essentially a mathematical ruler used to measure the "size" or "length" of a vector in a space X . However, just as you can measure a table in inches or centimeters, mathematicians can measure vector spaces using different norms.

A critical question arises: Do different norms change the fundamental nature of the space? For finite-dimensional spaces like $\mathbb{R}^2$, the answer is a comforting "No". We call these Equivalent Norms. According to Kreyszig [4] (Definition 2.4-4), two norms $\|\cdot\|$ and $\|\cdot\|_0$ are equivalent if they satisfy the following inequality for some positive constants a and b :

$$a\|x\|_0 \leq \|x\| \leq b\|x\|_0 \quad (1.3)$$

This ensures that if a vector is considered "small" by one norm, it is also "small" by the other [4]. While the specific "shape" of the distance might change, the underlying topology—the open sets and convergent sequences—remains identical.

1.6 The l_p Unit Ball: From Diamonds to Squares

To visualize this, we look at the l_p unit ball, defined by the set of points (x,y) such that:

$$|x|^p + |y|^p = 1 \quad (1.4)$$

As we vary the value of p , the geometry of the "unit circle" undergoes a fascinating transformation:

- $p = 1$: The unit ball is a diamond (the Manhattan norm).
- $p = 2$: The unit ball is the familiar Euclidean circle.
- $p \rightarrow \infty$: The unit ball expands into a square (the Supremum norm).

Our animation will show a single point moving around these boundaries, demonstrating how the "usual circle" is continuously warped into different l_p shapes while maintaining the same topological properties.

1.7 The Parametrization Strategy

To animate this transition smoothly, we cannot use the standard $x^2 + y^2 = 1$ logic. We need a general parametrization that works for any value of p . We use the following

transformation based on the sign function (sgn) to ensure we cover all four quadrants:

$$x(t) = \text{sgn}(\cos t) |\cos t|^{2/p} \quad (1.5)$$

$$y(t) = \text{sgn}(\sin t) |\sin t|^{2/p} \quad (1.6)$$

This ensures that $|x(t)|^p + |y(t)|^p = |\cos t|^2 + |\sin t|^2 = 1$, perfectly placing every point on the boundary for any p .

1.8 Implementation: The Norm Animation Engine

The following Python code [1] implements this warping geometry. It uses a dual-loop logic: one loop to change the value of p and another to trace the boundary dynamically via Matplotlib [2].

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 import matplotlib.animation as animation
4
5 # Define the p-values we want to visualize
6 p_values = [1, 2, 3, 5, 10, 20, np.inf]
7 points_per_norm = 200
8
9 def lp_unit_ball(p, num_points=200):
10     t = np.linspace(0, 2*np.pi, num_points)
11     if p == np.inf:
12         # Special case for the square (L-infinity)
13         x = np.array([1]*50 + list(np.linspace(1,-1,50)) + [-1]*50 +
14                        list(np.linspace(-1,1,50)))
15         y = np.array(list(np.linspace(-1,1,50)) + [1]*50 + list(np.
16                        linspace(1,-1,50)) + [-1]*50)
17         return x, y
18
19     # General p-norm parametrization
20     x = np.sign(np.cos(t)) * np.abs(np.cos(t))**(2/p)
21     y = np.sign(np.sin(t)) * np.abs(np.sin(t))**(2/p)
22     return x, y
23
24 fig, ax = plt.subplots(figsize=(6,6))
25 line, = ax.plot([], [], 'r-', lw=2)
26 point, = ax.plot([], [], 'bo')
```

```

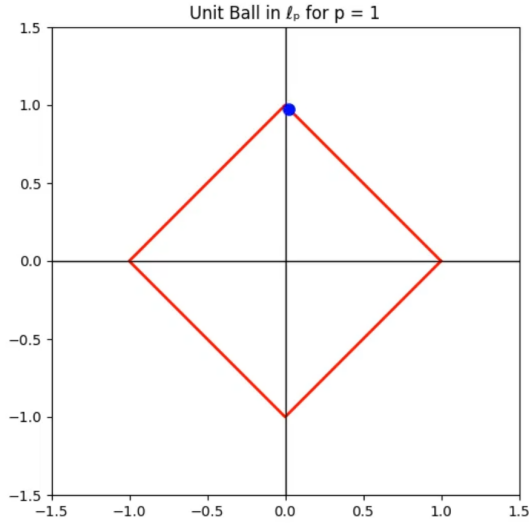
26 def update(frame):
27     # Determine which p-value and which point on the curve we are at
28     p_idx = (frame // points_per_norm) % len(p_values)
29     t_idx = frame % points_per_norm
30
31     p = p_values[p_idx]
32     x_full, y_full = lp_unit_ball(p)
33
34     line.set_data(x_full, y_full)
35     point.set_data([x_full[t_idx]], [y_full[t_idx]])
36     ax.set_title(f"L_p Unit Ball (p = {p})")
37     return line, point
38
39 ani = animation.FuncAnimation(fig, update, frames=len(p_values)*
    points_per_norm, interval=20)
40 plt.show()

```

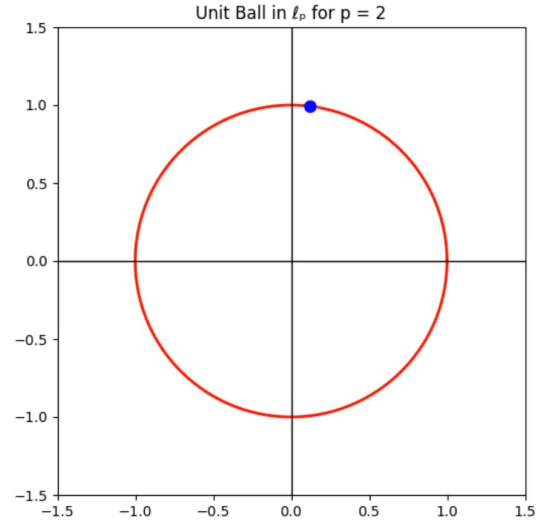
Listing 1.2: Animating Equivalent Norms in $\mathbb{R}^2$

1.8.1 Understanding the Code Logic

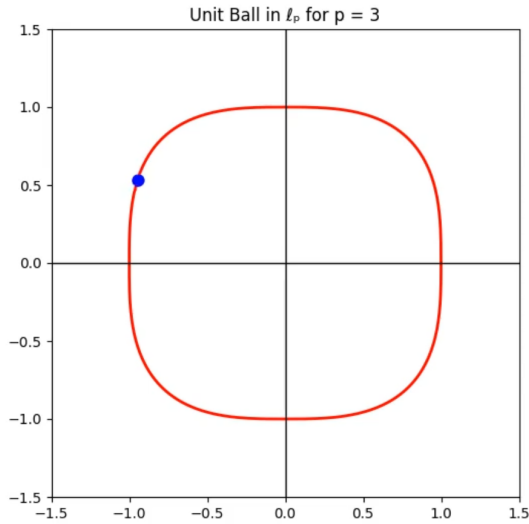
- The **lp_unit_ball** Function: This converts our mathematical parametrization into NumPy arrays. For the L_∞ norm, we handle it as a special case to ensure the square edges are perfectly sharp.
- The Frame Management: In the **update** function, we use the floor division (`//`) and modulo (`%`) operators. This allows us to cycle through multiple p values in a single continuous animation run.
- Dynamic Titles: We use f-strings (`f"L_p Unit Ball (p = {p})"`) to update the title in real-time, helping the viewer track which norm is currently being displayed.



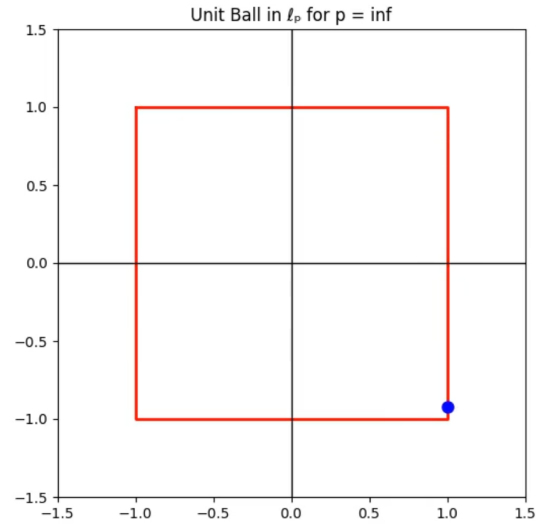
(a) $p = 1$: The Diamond Shape.



(b) $p = 2$: The Standard Circle.



(c) $p = 3$: Warping towards a square.



(d) $p = \infty$: The Square Boundary.

Figure 1.2: Visual progression of the ℓ_p unit ball as p increases.

1.9 Plotting Implicit Functions: The Ellipse

After understanding basic circles, we extend our animation engine to the ellipse. The general implicit equation of an ellipse centered at (h, k) with a semi-major axis a and semi-minor axis b is:

$$\frac{(x-h)^2}{a^2} + \frac{(y-k)^2}{b^2} = 1 \quad (1.7)$$

Just as we saw with earlier geometries [3], plotting directly from the implicit form is difficult for computers because it requires computing multiple y-values for a single x-value, leading to broken lines and rendering issues.

Therefore, we parametrize the ellipse using an angle parameter t . This traces the ellipse smoothly using sine and cosine functions:

$$x(t) = h + a \cos(t) \quad (1.8)$$

$$y(t) = k + b \sin(t) \quad (1.9)$$

In the code implementation below, we use Python [1] and Matplotlib [2] to animate a single point traveling along this path. The main logic involves computing the parametric coordinates for each frame and updating the position of our point object continuously.

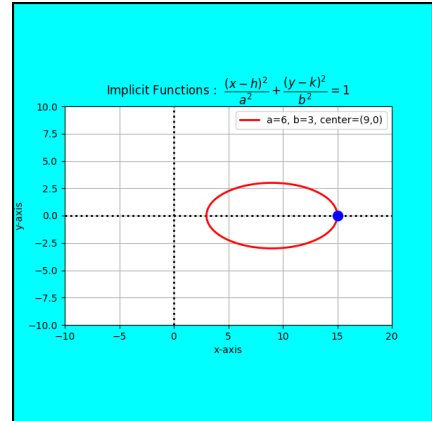


Fig: Parametric path of an ellipse.

Parametrization of General Shapes

- Line Segment (from (x_1, y_1) to (x_2, y_2)):

$$x(t) = x_1 + t(x_2 - x_1)$$

$$y(t) = y_1 + t(y_2 - y_1) \quad \text{for } t \in [0, 1]$$

- Parabola ($y^2 = 4ax$):

$$x(t) = at^2$$

$$y(t) = 2at \quad \text{for } t \in (-\infty, \infty)$$

- Hyperbola ($\frac{x^2}{a^2} - \frac{y^2}{b^2} = 1$):

$$x(t) = a \sec(t)$$

$$y(t) = b \tan(t) \quad \text{for } t \in [0, 2\pi)$$

Author Tip:

The transition from implicit to parametric forms is a universal strategy in computational geometry. For any general shape defined by $f(x, y) = 0$, finding a set of equations $x = g(t)$ and $y = h(t)$ allows us to map a single 1D time array t directly to 2D space. This avoids vertical line test failures and infinite slopes, making it the most robust mathematical approach for programmatic drawing and animation.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import matplotlib.animation as animation
4
5 def setup_axes():
6     fig, ax = plt.subplots(figsize=(6,6), dpi=100)
7     ax.set_xlim(-10, 20)
8     ax.set_ylim(-10, 10)
```

```

9     ax.axhline(0, color="black", linewidth=2, linestyle=":")
10    ax.axvline(0, color="black", linewidth=2, linestyle=":")
11    ax.set_aspect("equal", adjustable="box")
12    ax.grid(True)
13    ax.set_title("Implicit Functions : Ellipse")
14    return fig, ax
15
16    def starting_position():
17        point.set_data([], [])
18        return point,
19
20    def update_frame(frame, a, b, h, k):
21        # Main logic: Update point position using parametric equations
22        point.set_data([h + a*np.cos(frame)], [k + b*np.sin(frame)])
23        return point,
24
25    # Ellipse parameters
26    a, b = 6, 3
27    h, k = 9, 0
28
29    set_of_points = np.linspace(0, 2*np.pi, 300)
30    figure, ax = setup_axes()
31
32    # Plot the static path
33    ax.plot(h + a*np.cos(set_of_points), k + b*np.sin(set_of_points),
34           linewidth=2, color="red")
35    point, = ax.plot([], [], "bo", markersize=10)
36
37    ani = animation.FuncAnimation(
38        fig=figure, func=update_frame, frames=set_of_points,
39        fargs=(a,b,h,k), init_func=starting_position,
40        interval=30, blit=True)
41
42    plt.show()

```

Listing 1.3: Animating a Point on an Ellipse

1.10 Stochastic Geometry: The 2D Random Walk

Moving beyond deterministic geometric shapes, mathematical animation can also visualize probabilistic systems. A 2D random walk models a path consisting of a succession of random steps on a mathematical space.

The main logic in the Python [1] script relies on generating a random integer between 1 and 4 for every frame. Depending on the result, the current X or Y coordinate is incremented or decremented by 1. These new coordinates are appended to an array, and Matplotlib [2] continuously redraws the line to show the growing, randomized trail over time.

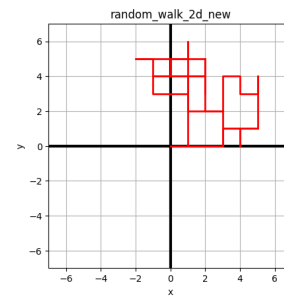


Fig: 2D Random Walk Path.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import matplotlib.animation as animation
4
5 fig, axs = plt.subplots()
6 axs.set_xlim(-7, 7)
7 axs.set_ylim(-7, 7)
8 axs.axhline(0, color='black', linewidth=3)
9 axs.axvline(0, color='black', linewidth=3)
10 axs.set_aspect('equal', adjustable='box')
11
12 x, y = [0], [0]
13 path_line, = axs.plot([], [], 'r--', linewidth=2)
14
15 def init():
16     x.clear()
17     y.clear()
18     x.append(0)
19     y.append(0)
20     path_line.set_data([], [])
21     return path_line,
22
23 def update(frame):
24     xold = x[-1]
25     yold = y[-1]
26
27     # Main logic: Pick a random direction (1: Right, 2: Left, 3: Up,
```

```

    4: Down)
28  rnd = np.random.randint(1, 5)
29  if rnd == 1:
30      xnew, ynew = xold + 1, yold
31  elif rnd == 2:
32      xnew, ynew = xold - 1, yold
33  elif rnd == 3:
34      xnew, ynew = xold, yold + 1
35  else:
36      xnew, ynew = xold, yold - 1
37
38  x.append(xnew)
39  y.append(ynew)
40  path_line.set_data(x, y)
41  return path_line,
42
43  ani = animation.FuncAnimation(fig, update, frames=100,
44                                init_func=init, blit=True, interval
45                                =50)
46  plt.title("2D Random Walk")
47  plt.grid(True)
48  plt.show()

```

Listing 1.4: Simulating a 2D Random Walk

1.11 Dynamic Transformations: Rotating a Curve

Mathematical animation is uniquely suited to display linear transformations, such as rotations, as a continuous motion rather than a simple "before and after" state. In this example, we apply a rotation matrix to a standard sine curve.

The core logic requires defining our base curve coordinates (x_c and y_c). For each frame, we treat the frame parameter k as our rotation angle θ . We then apply the standard 2D rotation matrix [3] to every point on the curve simultaneously:

$$x = x_c \cos(k) - y_c \sin(k) \quad (1.10)$$

$$y = x_c \sin(k) + y_c \cos(k) \quad (1.11)$$

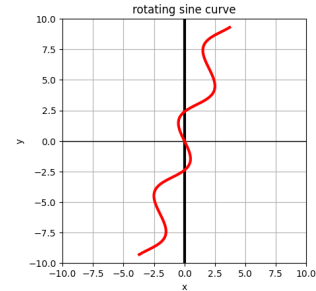


Fig: Rotating Sine Curve.

By feeding these updated arrays into Matplotlib [2], the entire curve appears to spin smoothly around the origin.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import matplotlib.animation as animation
4
5 fig, axs = plt.subplots()
6 axs.set_xlim(-10, 10)
7 axs.set_ylim(-10, 10)
8 axs.axhline(0, color='black', linewidth=1)
9 axs.axvline(0, color='black', linewidth=1)
10 axs.set_aspect('equal', adjustable='box')
11
12 # Time frames act as our rotation angle
13 t = np.linspace(-np.pi/2 + 0.01, np.pi/2 - 0.01, 100)
14 # Base coordinates for the sine curve
15 u = np.linspace(-10, 10, 100)
16
17 straight_line, = axs.plot([], [], linestyle='--', color='red',
18                             linewidth=3)
19
20 def init():
21     straight_line.set_data([], [])
22     return straight_line,
```

```

22
23 def update(frame):
24     k = frame # Current rotation angle
25     xc = u
26     yc = np.sin(u)
27
28     # Main logic: Apply 2D rotation matrix to the coordinates
29     x = xc*np.cos(k) - yc*np.sin(k)
30     y = xc*np.sin(k) + yc*np.cos(k)
31
32     straight_line.set_data(x, y)
33     return straight_line,
34
35 ani = animation.FuncAnimation(fig, update, frames=t,
36                               init_func=init, blit=True, interval
37                               =50)
38
39 axs.set_title("Rotating Sine Curve")
40 axs.grid(True)
41 plt.show()

```

Listing 1.5: Animating a Rotating Sine Curve

1.12 Further Explorations and Open Source Contributions in 2d

The animations detailed in this chapter—from the sliding ladder and equivalent norms to parametric ellipses and random walks—represent only a fraction of what is possible using this computational approach.

The complete suite of two-dimensional mathematical animations is actively maintained in a dedicated open-source repository. Beyond the scripts discussed, the repository contains several other foundational animations, including:

- **`amplitude_of_sine_wave.py`**: Visualizing how changing the amplitude coefficient dynamically stretches and compresses a waveform.
- **`unit_circle_anim.py`**: The foundational script demonstrating the parametric generation of a standard circle using sine and cosine over time.

Explore the Repository & Contribute!

All 2D scripts are publicly available in the `2d-anim` directory of the official project repository. You are encouraged to view, run, and modify these files to build your own mathematical intuition.

[View the 2D Animations Folder on GitHub](https://github.com/Shubham18024/Python-Mathematical-Animation/tree/main/2d-anim)

<https://github.com/Shubham18024/Python-Mathematical-Animation/tree/main/2d-anim>

Call for Contributions:

Mathematics is a collaborative discipline. If you have an idea for a new geometric visualization, a physics simulation, or an improvement to an existing script, you are welcome to fork the repository, write your animation, and submit a Pull Request!

1.13 Project Environment and Dependency Management using UV

To ensure complete reproducibility and efficient dependency management across different machines, this entire project is managed using `uv` [5]. Developed by Astral, `uv` is an extremely fast Python package and project manager written in Rust. It is designed as a modern, high-performance drop-in replacement for standard tools like `pip`, `virtualenv`, and `pip-tools`.

When dealing with mathematical animations, version control of libraries is critical. A slight update in how `Matplotlib` handles rendering backends or how `NumPy` processes specific array broadcasting can break an animation script that previously worked perfectly. By utilizing `uv`, we maintain a strict `pyproject.toml` and `uv.lock` file. This guarantees that anyone who clones this repository will build the exact same isolated virtual environment, instantly, without dealing with version conflicts or broken dependencies.

1.13.1 Workflow 1: Cloning and Running the Existing Project

For educators, evaluators, or students who simply want to view the animations locally, `uv` makes the setup process virtually instantaneous. You do not need to manually create virtual environments or guess which library versions to install.

Terminal Commands to Run the Project:

```
# 1. Clone the official GitHub repository
git clone https://github.com/Shubham18024/Python-Mathematical-Animation.git
cd Python-Mathematical-Animation

# 2. Sync the project (uv creates a virtual environment and
# installs the exact dependencies from uv.lock instantly)
uv sync

# 3. Run any animation script safely inside the isolated environment
uv run 2d-anim/scripts/cat_at_middle_ladder.py
```

1.13.2 Workflow 2: Starting a New Animation Project from Scratch

If you are inspired to create your own repository of mathematical animations, `uv` simplifies the initialization phase [5]. Instead of juggling `python -m venv venv` and `pip install`,

you can scaffold a robust project in seconds.

Terminal Commands for New Contributors:

```
# 1. Initialize a new Python project (creates pyproject.toml)
uv init my_math_animations
cd my_math_animations

# 2. Add the core mathematical and plotting libraries
# This automatically updates pyproject.toml and installs them
uv add numpy matplotlib

# 3. Execute your new script without activating environments
uv run main.py
```

The use of `uv` in this project not only demonstrates modern software engineering practices but also ensures that the focus remains entirely on the mathematics and logic, rather than wasting time debugging Python environments.

1.14 Expanding into the Third Dimension

While this current work focuses heavily on two-dimensional parametrizations and animations, the true power of this computational approach is unleashed when we move into 3D space. In our future work, we plan to extend this visualization framework towards fully three-dimensional problems. By utilizing X , Y , and Z coordinate transformations, we can animate complex surfaces like the Torus, Catenoid, and Helicoid. This will involve applying rotation matrices and translating coordinate points to manipulate 3D shapes programmatically, offering a profound way to teach multivariable calculus and spatial geometry.

1.15 Official Code Repository

All the core foundations, current 2D scripts, and upcoming 3D algorithms are actively maintained and available for experimentation. Students, educators, and developers can access the complete source code, track the project's evolution, and view the future 3D implementations at the official GitHub repository:

Project GitHub Repository:

<https://github.com/Shubham18024/Python-Mathematical-Animation>

Bibliography

- [1] Python Software Foundation. (2026). Python Language Reference. Retrieved from <https://docs.python.org/3/>
- [2] Matplotlib Developers. (2026). Matplotlib Animation Guide. Retrieved from <https://matplotlib.org/>
- [3] Loney, S. L. (1895). The Elements of Coordinate Geometry. London: Macmillan and Co.
- [4] Kreyszig, Erwin (1989). Introductory Functional Analysis with Applications (1st ed.). John Wiley & Sons. Wiley-India Student Edition. Indian Reprint 2007.
- [5] Astral. (2024). uv: First steps. Retrieved from <https://docs.astral.sh/uv/getting-started/first-steps/>